QCon
全球软件开发大会

InfoQ 技术大会

AI Coding 在大型客户端工程中的落地实践——从通才模型到专才 Agent

拓海 | 小红书客户端基础技术组



同一个模型，同一份需求，两个结果

图片选择页：最多选 9 张

第一版 · 整页交给一个 Agent

第 10 张仍然能选

- 需求里写了上限，Agent 也复述过
- 后续修布局、补 UI 状态，规则被忽略
- 编译通过，数量限制却没有生效

第二版 · 按职责拆开，分别检查

功能点完成度 100%

- 图片项、选择栏、空态，分别交给 Agent
- 每个 Agent 只拿当前任务需要的信息
- 完成后，分别检查各部分的实现

（人工检测结果）

任务怎么拆、每一步怎么检查，都会影响结果

复杂任务中反复出现的三个问题

任务越复杂、修改越多，越容易出现

01 上下文爆炸 · 细节丢失

需求和设计稿都给了，
关键规则却没有落到代码里

任务边界与信息取舍不清

02 长程编码 · 目标漂移

为了解决局部问题，
改动逐渐超出原任务范围

阶段反馈后置，问题难归因

03 缺少运行反馈

编译通过，交互未必正确，
出了问题难以定位

运行证据未接入

类比：再优秀的工程师，被绑住手、不给调试环境、还要短时间交付，也很难保证质量

从工程原因出发，组织协作与反馈

借鉴团队开发的做法，把职责、交接和检查安排清楚。

上下文爆炸		长程 · 目标漂移	缺少运行反馈
拆解 做什么 职责完整、可独立推理的节点 明确输入、输出与交互约定	**隔离** 控制影响范围 独立 SubAgent、独立上下文 限定改动范围，便于局部修复	**约束** 交什么，能改哪 用结构化产物明确约定与范围 每次交接检查，减少偏差累积	**测试闭环** 怎么算对 写代码时测逻辑， 运行后验行为 发现问题后修复，再次验证

把任务边界落到职责、上下文、交接和验证里。

01

拆 解

让每一步都可独立推理

拆解粒度

按功能拆

任务可能偏大

一个功能可能跨多个文件
仍需同时处理多项判断

按文件拆

职责可能被拆散

一项职责分在多个文件
分别交接，增加配合成本

按节点拆

✓ 职责完整

输入输出与协作约定明确
可独立推理的任务单元

Terpsi：小红书为适配 AI 出码设计的页面架构

节点怎样设计，节点之间怎样协作?

Terpsi：小红书的节点化页面架构



页面与操作示意

页面父节点
页面协调范围

关注引导子节点
引导展示区域

点赞子节点
按钮与点赞状态

Terpsi 节点的协作约定

数据和事件对外协作，状态留在节点内部

DATA	addChild + dataMapper 传入数据
STATE	ViewModel 内部维护，Element 监听
EVENT	输出事件报告结果，输入事件接收动作

父节点协调

```
like.event.onLike
    .onEach {
        if (shouldShowGuide())
            guide.event.followGuide.emit(Unit)
    }
    .launchIn(coroutineScope)
```

Agent 实现本节点，父节点负责节点间协作

节点说明与骨架生成

同一组点赞、关注引导与父节点，逐步落到代码结构

节点描述

node_spec.json

```
{
  "blockName": "NoteDetail",
  "blockType": "Ui-Block",
  "responsibilities": "协调子节点",
  "childBlocks": [
    {
      "blockName": "Like",
      "blockType": "Ui-Block",
      "responsibilities": "点赞并上报"
    },
    {
      "blockName": "FollowGuide",
      "blockType": "Ui-Block",
      "responsibilities": "展示关注引导"
    }
  ]
}
```

提取骨架 JSON

生成器输入

```
{
  "name": "NoteDetail",
  "type": "normal",
  "children": [
    {
      "name": "Like",
      "type": "normal",
      "children": []
    },
    {
      "name": "FollowGuide",
      "type": "normal",
      "children": []
    }
  ]
}
```

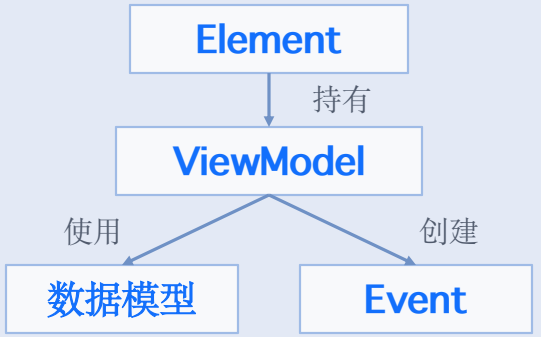
按规则提取　名称、类型、父子层级

脚本生成

TerpsiCreator Script

按平台模板生成

每个节点的类骨架



生成路径回填 node_spec.json，职责说明供后续实现使用

节点实现：补齐业务 + 串联节点

拆分 Agent
节点树与职责

→

点赞节点	SubAgent A
关注引导节点	SubAgent B

→

页面父节点
SubAgent C

SubAgent 返回

```
{
  "status": "completed",
  "task_name": "Like",
  "changed_files": [
    ".../LikeViewModel", ...
  ],
  "output_dir": ".../nodes/Like/"
}
```

返回摘要：改动文件的路径列表

节点实现产物：impl_result.json

```
{
  "task_name": "Like",
  "changed_files": [{
    "path": ".../LikeViewModel",
    "change_type": "filled_skeleton",
    "description": "点赞成功后上报 onLike"
  }]
}
```

落盘明细：记录文件及具体改动

仅重试未通过的节点，本层全部通过后统一更新 meta.json

Agent 按职责补充业务方法、状态和事件

02

隔　离

尽量把错误的影响控制在局部

任务分开，上下文也要分开

每个 Agent 接收必要信息，产物检查通过后再交接

同一段上下文，连续执行

需求、前面讨论、实现尝试
持续积累在同一段对话里



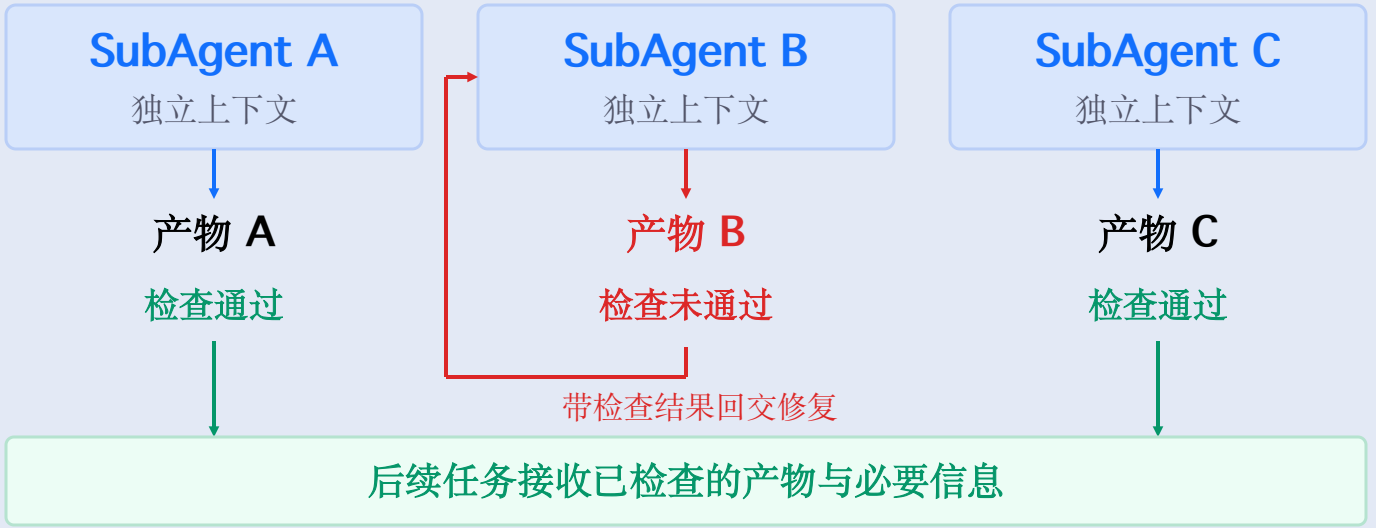
错误判断留在上下文里
后续步骤可能直接沿用

步骤分开了，上下文仍然共用

独立上下文，按产物交接

分别带入：任务说明、必要代码、外部接口与改动范围

不传：其他任务的整段对话、已经否定的猜测



独立上下文减少干扰，产物检查不通过就先修复再交接

角色分工与协作原则

主 Agent 负责调度和检查，SubAgent 与固定工具分别执行

主 Agent

分配任务、跟踪状态
检查交付结果

SubAgent

完成当前任务
交回产物，反馈问题

固定工具

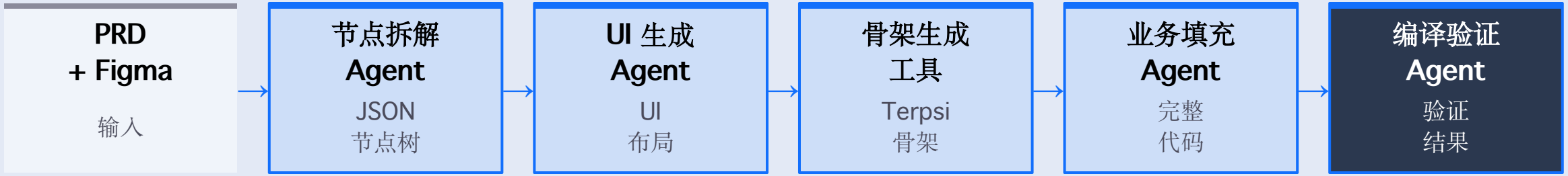
按既定规则执行
例如由节点 JSON 生成骨架

主 Agent 不接管实现	避免各任务的实现细节，重新堆进同一段上下文
跨节点修改先反馈	由主 Agent 协调，避免为了当前任务改坏其他节点
交接前写清任务说明	职责、交付物和改动范围，都要能对照检查

分工有明确依据，出问题才知道该交回给谁

分阶段流水线：隔离如何在流程中落地

沿着点赞节点，查看每个阶段交回的产物



final_review/review_result.json

Review Agent · 全量审查

```
{
  "scope": "full_project",
  "has_issues": false,
  "issues": [],
  "review_verdict": "PASS"
}
```

审查结论与编译结果分别记录

compile_check/result.json

编译 Agent · 编译检查

```
{
  "status": "success",
  "total_rounds": 1,
  "fixed_issues": []
}
```

构建日志另存 build_round_N.log

示例为字段节选，路径与结果用于演示；执行进度另记 meta.json

03

约 束

把已经确认的内容，变成后续实现的依据

约束的边界：哪些固定，哪些留给模型

约束过松

结构和接口反复改
容易顺手改到别的节点
前后实现对不上

← 太松

适度约束

职责、接口和改动范围固定
节点内部的实现可以发挥
交接时检查有没有越界

✓ 适度

约束过紧

必要的状态和事件也不让补
规则都遵守了
业务却做不完整

太紧 →

经常返工的地方补规则，挡住正常实现的限制及时调整

四层产物，把已经确认的内容留下来

交付物 / 负责人	已经确定的内容	后续修改边界
节点 JSON 拆解 Agent	节点划分与交互约定	只定义节点和接口，不写实现 后续不擅自增删或合并节点
UI 文件 UI Agent	界面层级与组件边界	业务逻辑不混进 UI 页面协调逻辑留在父节点
Terpsi 骨架 固定工具	节点接口与调用结构	工具按规则生成，模型不手写接口 接口与业务实现保持分离
业务逻辑 节点 Agent	沿用已确认的职责、接口和骨架	仅在预留位置补齐业务逻辑 结构或接口有问题，反馈后调整

已经确认的内容继续沿用，确实需要调整时回到对应阶段

「完成」要有明确的检查标准

根据交付物选择检查项，通过以后再交给下一步

1

逻辑单测

有业务逻辑的节点
输入、输出和状态变化

2

产物完整性

节点 JSON、UI 和骨架
格式、字段和节点是否齐全

3

任务状态一致性

任务记录与实际产物
是否真的生成并交回

4

改动范围检查

查看实际代码改动
有没有越界、改动骨架

检查不通过，就交回负责的 Agent 定向修复

主 Agent 记录结果，修复后重验。仍无法通过，交给人工判断

04

测试闭环

从「编译通过」到「运行正确」——出码中单测 + 出码后自动判定修复

节点逻辑单测

按节点验证输入输出约定，把业务规则写成可执行的检查

节点出码

SubAgent 生成
节点代码

生成单测

基于节点输入输出
契约自动生成用例

运行单测

纯逻辑层执行
不依赖 UI 环境

判定

通过 → 进入下一层
失败 → 本节点闭环修复

测试用例（伪代码）：数量上限为 9 张

第 9 张，可以选

```
@Test
fun canSelect9() {
  val s = selected(1..8)
  check(s.select(9))
  check(s.ids == ids(1..9))
}
```

第 10 张，不能选

```
@Test
fun reject10() {
  val s = selected(1..9)
  check(!s.select(10))
  check(s.ids == ids(1..9))
}
```

取消后，可以继续选

```
@Test
fun canSelectAgain() {
  val s = selected(1..9)
  s.unselect(9)
  check(s.select(10))
  check(s.ids == ids(1..8,10))
}
```

测试 case 自动运行与判定修复

按操作采集证据，对照预期完成修复与复验



截图 / UI Tree / 日志

```
{
  "case_id": "TC-001",
  "title": "九张上限",
  "preconditions": {"test_data": "图片选择页已选 9 张"},
  "steps": [{
    "no": 3,
    "action": "点击第 10 张图片",
    "expect_nl": "原 9 张保持选中，并提示上限"
  }]
}
```

preconditions
运行前，先准备好状态

action
这一步，具体做什么

expect_nl
做完以后，应该是什么结果

cases.json 单个用例节选：九张场景为演示，前置步骤及其他字段省略

案例，以及这套方法不解决什么

UI 与逻辑为人工体验评分，耗时包含人工修改

案例	耗时	UI	逻辑	编译
图片选择器	2h+	80%	100%	一次通过
图片查看器	2h+	80%	90%	一次通过
拍照页	1h+	100%	100%	一次通过

鸿蒙跨端翻译

多项 Android → 鸿蒙 ArkTS 翻译需求，经同一套流程交付。

仍需人工判断

- 设计还原中的视觉细节
- 需求歧义与验收标准
- 跨节点的结构与接口调整
- 自动化未覆盖或无法可靠判定的场景

需求判断、视觉验收和复杂问题处理，仍然需要人来把关

实践中的三条经验

01

约束要适度，也要调整

锁定职责与协议，保留实现空间
结合实际错误调整规则

02

中间产物也是交付物

节点树、UI 与骨架逐层交付
保存、检查，并能追溯到任务

03

验证贯穿交付过程

交接查产物，节点测逻辑
运行看实际表现
用验证结果判断是否完成

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

大模型不是银弹，工程体系才是护城河

从「一个模型解决一切」到「专才协同、层层收口」

让模型能力在可检查、可修复的工程体系中持续发挥

拆解 + 隔离 + 约束 + 测试闭环



— Thank You —